

안드레이 카파시가 제안한 LLM Wiki가 원지와 실제 구축하는 단계별 내용을 상세하게 설명해 줘(Windows와 Mac 환경 각각)

안드레이 카파시의 "LLM Wiki" 개념

LLM Wiki는 카파시가 2026년 4월 4일 GitHub Gist(llm-wiki.md)로 공개한 아이디어로, 완성된 제품이나 코드가 아니라 기존 RAG 방식처럼 매번 원본 문서에서 정보를 재검색하는 대신, LLM이 큐레이션된 소스 자료를 점진적으로 지속적이고 상호연결된 마크다운 위키로 컴파일하도록 하는 "아이디어 파일"입니다.

핵심 개념

대부분의 RAG 방식(NotebookLM, ChatGPT 파일 업로드 등)은 문서를 업로드하면 질의 시점에 관련 조각을 검색해서 답을 생성하는데, 이 경우 LLM은 매 질문마다 지식을 처음부터 재발견하게 되어 축적이 일어나지 않습니다. LLM Wiki는 다릅니다. 새 소스를 추가하면 LLM이 단순히 색인만 하는 게 아니라 내용을 읽고 핵심 정보를 추출해 기존 위키에 통합합니다 — 엔티티 페이지를 업데이트하고, 주제 요약을 수정하고, 새 데이터가 기존 주장과 모순되는 부분을 표시하고, 종합(synthesis)을 강화하거나 도전합니다. 즉 지식이 한 번 컴파일되고 계속 최신 상태로 유지되는 것이지, 매 질의마다 다시 도출되는 게 아닙니다.

3단계 아키텍처

- **Raw sources(원본 소스):** 논문, 기사, 이미지, 데이터 파일 등 큐레이션된 원본 문서 모음. LLM은 이를 읽기만 하고 절대 수정하지 않습니다 — 진실의 원천(source of truth)입니다.
- **The wiki(위키):** LLM이 생성한 마크다운 파일들의 디렉터리. 요약, 엔티티 페이지, 개념 페이지, 비교 페이지, 개요, 종합 페이지 등. LLM이 이 계층을 전적으로 소유하며 페이지를 생성·업데이트하고 상호참조를 유지합니다.
- **The schema(스키마):** Claude Code용 CLAUDE.md나 Codex용 AGENTS.md 같은 문서로, 위키 구조와 규칙, 그리고 소스 수집(ingest)·질의응답·유지보수 시의 워크플로우를 LLM에게 알려줍니다.

3가지 운영(Operations)

Ingest(수집): 새 소스를 raw 컬렉션에 넣고 LLM에게 처리를 지시합니다. LLM이 소스를 읽고 핵심 내용을 논의한 뒤 위키에 요약 페이지를 작성하고, 색인을 갱신하고, 관련 엔티티·개념 페이지들을 업데이트하고, 로그에 항목을 추가합니다. 소스 하나가 10~15개 위키 페이지에 영향을 줄 수 있습니다.

Query(질의): 위키에 질문을 던지면 LLM이 관련 페이지를 찾아 읽고 인용과 함께 답을 종합합니다. 좋은 답변은 다시 위키에 새 페이지로 파일링될 수 있어, 대화 기록 속으로 사라지지 않고 지식베이스에 축적됩니다.

Lint(점검): 주기적으로 페이지 간 모순, 새 소스로 인해 낡아진 주장, 인바운드 링크가 없는 고아 페이지, 언급되었지만 전용 페이지가 없는 중요 개념, 누락된 상호참조 등을 점검합니다.

여기에 **index.md**(콘텐츠 중심 카탈로그, 각 페이지별 링크와 한 줄 요약)와 **log.md**(연대순 기록, 언제 어떤 ingest/query/lint가 있었는지)라는 두 개의 특수 파일이 위키 탐색을 돕습니다.

단계별 구축 가이드

Claude Code를 에이전트로 사용하는 것을 전제로 설명드리겠습니다 (Codex 등 다른 에이전트도 동일한 패턴 적용 가능).

공통 준비물

- Claude Code (Node.js 기반 CLI)
- Obsidian (위키를 눈으로 보는 뷰어, 선택이지만 강력 추천)
- Git (버전 관리, 선택이지만 강력 추천)

Windows 환경

1단계 — Node.js 설치

```
winget install OpenJS.NodeJS.LTS
```

설치 후 새 PowerShell 창에서 `node -v`로 확인합니다.

2단계 — Claude Code 설치

```
npm install -g @anthropic-ai/claude-code
```

3단계 — 프로젝트 폴더 생성

```
mkdir C:\claude\projects\my-wiki
```

```
cd C:\claude\projects\my-wiki
```

```
mkdir raw
```

mkdir wiki

4단계 — Git 초기화 (선택)

git init

5단계 — Obsidian 설치 공식 사이트(obsidian.md)에서 Windows 인스톨러를 받아 설치한 뒤, "Open folder as vault"로 C:\Claude\projects\my-wiki를 엽니다. 이후 이미지 자동 다운로드를 원하시면 Settings → Files and links에서 "Attachment folder path"를 raw/assets/로 고정하고, Settings → Hotkeys에서 "Download attachments for current file"에 단축키(예: Ctrl+Shift+D)를 지정해두시면 편리합니다.

6단계 — Claude Code 실행 및 스키마 생성

claude

그 다음 카파시의 원본 Gist 내용을 통째로 붙여넣고 이렇게 요청합니다:

"이 아이디어 파일을 읽고 이 디렉터리에 LLM Wiki를 구축하도록 도와줘. 시작하기 전에 이 위키가 무엇에 관한 것인지, 어떤 소스를 넣을 계획인지 먼저 물어봐줘. 답변을 들으면 그에 맞는 CLAUDE.md 스키마 파일을 작성해줘."

Claude가 질문을 던지면 답변하고, CLAUDE.md, wiki/index.md, wiki/log.md가 생성되는 것을 확인합니다.

7단계 — 첫 소스 수집(ingest) raw/ 폴더에 첫 문서(PDF, txt, md 등)를 넣고 Claude Code에게 "이 소스를 수집해줘"라고 요청합니다.

Mac 환경

1단계 — Homebrew 설치 (없다면)

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2단계 — Node.js 설치

brew install node

3단계 — Claude Code 설치

npm install -g @anthropic-ai/claude-code

4단계 — 프로젝트 폴더 생성

mkdir -p ~/claude/projects/my-wiki/raw

```
mkdir -p ~/claude/projects/my-wiki/wiki
```

```
cd ~/claude/projects/my-wiki
```

5단계 — Git 초기화 (선택)

```
git init
```

6단계 — Obsidian 설치

```
brew install --cask obsidian
```

실행 후 "Open folder as vault"로 ~/claude/projects/my-wiki를 선택합니다. 이미지 자동 다운로드 설정은 Windows와 동일합니다.

7단계 — Claude Code 실행 및 스키마 생성

```
claude
```

이후 절차는 Windows와 완전히 동일합니다 — Gist 내용을 붙여넣고, 질문에 답하고, CLAUDE.md가 생성되는지 확인한 뒤 첫 소스를 raw/에 넣고 ingest를 요청합니다.

참고 팁

- Obsidian Web Clipper 브라우저 확장을 쓰면 웹 아티클을 마크다운으로 변환해 raw 폴더에 바로 저장할 수 있어 소스 수집이 빨라집니다.
- 위키가 커지면(수백 페이지 이상) 인덱스 파일만으로는 부족해질 수 있는데, 이때는 qmd 같은 로컬 하이브리드(BM25/벡터) 검색 도구를 CLI나 MCP 서버 형태로 붙이는 것을 고려하시면 됩니다.
- 카파시 본인의 위키는 약 100개 아티클, 40만 단어까지 커졌는데도 인덱스와 요약만으로 LLM이 효율적으로 탐색할 수 있었다고 언급했습니다. 이 정도 규모까지는 별도 인프라 없이 이 패턴만으로 충분합니다.

최상길님의 GD RDM 프로젝트나 KOSPI 모니터링 시스템에서 다뤄오신 MCP/Skills/CLAUDE.md 구조와 상당히 유사한 철학이라, 스키마 파일 설계 경험을 그대로 옮겨오실 수 있을 것 같습니다. 원하시면 GD RDM 맥락에 맞춘 CLAUDE.md 스키마 예시도 작성해 드릴 수 있습니다.

카파시의 원본 Gist 내용을 알려줘

죄송하지만, 카파시의 Gist 원문은 저작권이 있는 타인의 저작물이라 전체를 그대로("한 글자도 누락 없이") 복사해서 보여드릴 수는 없습니다. 대신 원문의 모든 섹션 내용을 빠짐없이, 제 자신의 표현으로 상세하게 풀어서 정리해 드리겠습니다. 원문이 필요하시면 아래 링크에서 직접 확인하실 수 있습니다.

원문 링크: <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f> (2026년 4월 4일 게시)

문서 구조 (원문의 모든 섹션)

제목 및 서두

문서 제목은 "LLM Wiki"이며, 부제로 "LLM을 이용해 개인 지식베이스를 구축하는 패턴"이라고 소개됩니다. 이 문서는 완성된 코드가 아니라 "아이디어 파일(idea file)"로 설계되었으며, OpenAI Codex, Claude Code, OpenCode/Pi 등 자신이 쓰는 LLM 에이전트에 그대로 복사-붙여넣기 하도록 만들어졌습니다. 목적은 고차원적인 아이디어를 전달하는 것이고, 세부 구현은 사용자와 에이전트가 협업하며 만들어가는 것입니다.

핵심 아이디어(The core idea)

대다수 사람들이 LLM과 문서를 다루는 경험은 RAG 방식입니다 — 파일 모음을 업로드하면 질의 시점에 관련 조각을 검색해서 답을 생성하는 방식입니다. 이 방식은 작동은 하지만, LLM이 매 질문마다 지식을 처음부터 재발견한다는 문제가 있습니다. 축적이 일어나지 않는 것입니다. 다섯 개의 문서를 종합해야 하는 미묘한 질문을 하면, LLM은 매번 관련 조각을 다시 찾아 짜맞춰야 합니다. 쌓이는 게 없습니다. NotebookLM, ChatGPT 파일 업로드, 대부분의 RAG 시스템이 이렇게 동작합니다.

여기서 제안하는 아이디어는 다릅니다. 질의 시점에 원본 문서에서 단순히 검색만 하는 대신, LLM이 **점진적으로 영구적인 위키를 구축하고 유지**합니다 — 사용자와 원본 소스 사이에 놓이는, 구조화되고 상호연결된 마크다운 파일 모음입니다. 새 소스를 추가하면 LLM은 단순히 나중에 검색하기 위해 색인만 하는 게 아니라, 그것을 읽고 핵심 정보를 추출해 기존 위키에 통합합니다 — 엔티티 페이지를 업데이트하고, 주제 요약을 수정하고, 새 데이터가 기존 주장과 어디서 모순되는지 기록하고, 발전해가는 종합을 강화하거나 도전합니다. 지식은 한 번 컴파일된 후 계속 최신 상태로 **유지**되는 것이지, 매 질의마다 다시 도출되는 게 아닙니다.

핵심 차이는 이것입니다: **위키는 영구적이고 복리로 쌓이는 산출물이다**. 상호참조는 이미

존재하고, 모순은 이미 표시되어 있으며, 종합은 이미 지금까지 읽은 모든 것을 반영하고 있습니다. 소스를 추가하고 질문을 던질 때마다 위키는 점점 더 풍부해집니다.

카파시 본인은 위키를 직접 쓰는 경우가 거의 없고, LLM이 모든 것을 작성하고 유지합니다. 본인은 소싱, 탐색, 좋은 질문을 던지는 것을 담당합니다. LLM이 요약, 상호참조, 파일링, 부기(bookkeeping) 등 시간이 지나도 지식베이스가 실제로 유용하게 유지되도록 만드는 모든 일을 다 합니다. 실제로는 한쪽에는 LLM 에이전트를, 다른 한쪽에는 Obsidian을 열어두고 작업한다고 설명합니다 — LLM이 대화를 바탕으로 편집하면 실시간으로 링크를 따라가고 그래프 뷰를 확인하고 업데이트된 페이지를 읽으며 결과를 살펴보는 방식입니다. "Obsidian이 IDE이고, LLM이 프로그래머이며, 위키가 코드베이스다"라는 비유를 듭니다.

이 패턴이 적용될 수 있는 다양한 맥락의 예시로 다음을 듭니다:

- **개인:** 자신의 목표, 건강, 심리, 자기계발을 추적 — 일기, 기사, 팟캐스트 노트를 정리하며 시간이 지남에 따라 자신에 대한 구조화된 그림을 구축
- **연구:** 몇 주에서 몇 달에 걸쳐 한 주제를 깊이 파고들기 — 논문, 기사, 보고서를 읽으며 점진적으로 종합적인 위키와 발전하는 논지를 구축
- **책 읽기:** 책을 진행하면서 파일링하고, 인물·주제·플롯 갈래와 그 연결에 대한 페이지를 구축. Tolkien Gateway 같은 팬 위키(수천 개의 상호연결된 페이지가 자원봉사자 커뮤니티에 의해 수년에 걸쳐 구축됨)를 예로 들며, 읽으면서 LLM이 상호참조와 유지관리를 모두 처리하는 개인용 버전을 만들 수 있다고 설명
- **비즈니스/팀:** Slack 스레드, 회의록, 프로젝트 문서, 고객 통화로 채워지는 LLM 유지 내부 위키. 사람이 업데이트를 검토하는 휴먼-인-더-루프 방식일 수도 있음
- **경쟁사 분석, 실사(due diligence), 여행 계획, 강의 노트, 취미 딥다이브** 등 시간에 걸쳐 지식을 축적하고 정리하고 싶은 모든 영역

아키텍처(Architecture) — 3계층

- **Raw sources(원본 소스):** 큐레이션된 소스 문서 모음(기사, 논문, 이미지, 데이터 파일). 이것들은 불변(immutable)입니다 — LLM은 여기서 읽기만 하고 절대 수정하지 않습니다. 이것이 진실의 원천(source of truth)입니다.
- **The wiki(위키):** LLM이 생성한 마크다운 파일 디렉터리(요약, 엔티티 페이지, 개념 페이지, 비교, 개요, 종합). LLM이 이 계층을 전적으로 소유합니다 — 페이지를 만들고, 새 소스가 들어오면 업데이트하고, 상호참조를 유지하고, 모든 것을 일관되게 유지합니다. 사용자는 읽고, LLM은 씁니다.

- **The schema(스키마):** Claude Code용 CLAUDE.md나 Codex용 AGENTS.md 같은 문서로, 위키가 어떻게 구조화되어 있는지, 어떤 관례를 따르는지, 소스를 수집하거나 질문에 답하거나 위키를 유지관리할 때 어떤 워크플로우를 따라야 하는지를 LLM에게 알려줍니다. 이것이 핵심 설정 파일입니다 — LLM을 일반 챗봇이 아니라 규율 있는 위키 관리자로 만드는 것이 바로 이 파일입니다. 사용자와 LLM이 자신의 도메인에 맞는 것을 알아가면서 시간이 지남에 따라 이 파일을 함께 발전시켜 나갑니다.

운영(Operations) — 3가지

Ingest(수집): 새 소스를 원본 컬렉션에 넣고 LLM에게 처리를 지시합니다. 예시 흐름은 다음과 같습니다 — LLM이 소스를 읽고, 핵심 요점을 사용자와 논의하고, 위키에 요약 페이지를 작성하고, 인덱스를 업데이트하고, 위키 전반에서 관련 엔티티·개념 페이지를 업데이트하고, 로그에 항목을 추가합니다. 소스 하나가 10~15개의 위키 페이지에 영향을 줄 수 있습니다. 카파시 개인적으로는 소스를 한 번에 하나씩 수집하며 계속 관여하는 것을 선호한다고 밝힙니다 — 요약을 읽고, 업데이트를 확인하고, 무엇을 강조할지 LLM에게 지시합니다. 하지만 감독을 덜 하면서 여러 소스를 한꺼번에 일괄 수집할 수도 있습니다. 자신의 스타일에 맞는 워크플로우를 개발해서 향후 세션을 위해 스키마에 문서화하는 것은 사용자 몫입니다.

Query(질의): 위키에 질문을 던집니다. LLM이 관련 페이지를 검색하고 읽어서 인용과 함께 답을 종합합니다. 답변은 질문에 따라 마크다운 페이지, 비교표, 슬라이드 텍(Marp), 차트(matplotlib), 캔버스 등 다양한 형태를 취할 수 있습니다. 중요한 통찰은 이것입니다: **좋은 답변은 다시 위키에 새 페이지로 파일링될 수 있다.** 요청한 비교, 분석, 발견한 연결고리 — 이런 것들은 가치가 있으므로 대화 기록 속으로 사라지면 안 됩니다. 이렇게 하면 탐색 결과가 수집된 소스와 마찬가지로 지식베이스에 복리로 쌓입니다.

Lint(점검): 주기적으로 LLM에게 위키의 건강 상태를 점검하도록 요청합니다. 확인할 것: 페이지 간 모순, 더 새로운 소스에 의해 대체된 낡은 주장, 인바운드 링크가 없는 고아 페이지, 언급되었지만 자신의 페이지가 없는 중요한 개념, 누락된 상호참조, 웹 검색으로 채울 수 있는 데이터 공백. LLM은 조사할 새로운 질문과 찾아볼 새로운 소스를 제안하는데 능합니다. 이것이 위키가 성장하면서도 건강하게 유지되도록 해줍니다.

인덱싱과 로깅(Indexing and logging)

위키가 커질 때 LLM(과 사용자)이 탐색하는 데 도움이 되는 두 개의 특수 파일이 있으며, 서로 다른 목적을 가집니다.

index.md는 콘텐츠 중심입니다. 위키 안의 모든 것에 대한 카탈로그로, 각 페이지가 링크, 한 줄 요약, 선택적으로 날짜나 소스 개수 같은 메타데이터와 함께 나열됩니다. 카테

고리(엔티티, 개념, 소스 등)별로 정리되며, LLM이 수집할 때마다 업데이트합니다. 질의에 답할 때 LLM은 먼저 인덱스를 읽어 관련 페이지를 찾은 다음 해당 페이지로 들어갑니다. 이 방식은 중간 규모(소스 약 100개, 페이지 수백 개)에서 놀라울 만큼 잘 작동하며, 임베딩 기반 RAG 인프라를 구축할 필요를 없애줍니다.

log.md는 시간순입니다. 언제 무슨 일이 있었는지(수집, 질의, 점검)에 대한 추가 전용 (append-only) 기록입니다. 유용한 팁으로, 각 항목이 일관된 접두사(예: ## [2026-04-02] ingest | Article Title)로 시작하면 로그를 grep 같은 간단한 유닉스 도구로 파싱할 수 있게 됩니다 — `grep "^## \w[" log.md | tail -5`로 최근 5개 항목을 볼 수 있습니다. 로그는 위키의 발전 타임라인을 제공하고 LLM이 최근에 무엇이 이루어졌는지 이해하도록 돕습니다.

선택사항: CLI 도구(Optional: CLI tools)

어느 시점에서는 LLM이 위키를 더 효율적으로 다룰 수 있도록 작은 도구를 만들고 싶어질 수 있습니다. 위키 페이지에 대한 검색 엔진이 가장 명백한 예입니다 — 소규모에서는 인덱스 파일만으로 충분하지만, 위키가 커지면 제대로 된 검색이 필요합니다.

qmd(tobi/qmd, 마크다운 파일용 로컬 검색 엔진으로 하이브리드 BM25/벡터 검색과 LLM 재순위화를 온디바이스로 제공)가 좋은 선택지로 제시됩니다. CLI(LLM이 셀아웃 가능)와 MCP 서버(LLM이 네이티브 도구로 사용 가능) 둘 다 있습니다. 더 간단한 것을 직접 만들 수도 있으며, 필요에 따라 LLM이 소박한 검색 스크립트를 "바이브 코딩"하도록 도울 수도 있습니다.

팁과 요령(Tips and tricks)

- **Obsidian Web Clipper**: 웹 아티클을 마크다운으로 변환해주는 브라우저 확장. 소스를 원본 컬렉션에 빠르게 넣는 데 매우 유용합니다.
- **이미지를 로컬에 다운로드하기**: Obsidian Settings → Files and links에서 "Attachment folder path"를 고정 디렉터리(예: raw/assets/)로 설정합니다. 그다음 Settings → Hotkeys에서 "Download"를 검색해 "Download attachments for current file"을 찾아 단축키(예: Ctrl+Shift+D)에 바인딩합니다. 아티클을 클립한 후 단축키를 누르면 모든 이미지가 로컬 디스크로 다운로드됩니다. 선택사항이지만 유용합니다 — 깨질 수 있는 URL에 의존하는 대신 LLM이 이미지를 직접 보고 참조할 수 있게 해줍니다. LLM은 인라인 이미지가 있는 마크다운을 한 번에 네이티브로 읽을 수 없다는 점에 유의해야 합니다 — 우회 방법은 LLM이 먼저 텍스트를 읽은 다음 참조된 이미지 일부 또는 전체를 별도로 보게 하여 추가 맥락을 얻는 것입니다. 다소 투박하지만 충분히 잘 작동합니다.
- **Obsidian의 그래프 뷰**: 위키의 형태를 보는 가장 좋은 방법입니다 — 무엇이 무

것에 연결되어 있는지, 어느 페이지가 허브인지, 어느 페이지가 고아인지.

- **Marp:** 마크다운 기반 슬라이드 덱 포맷. Obsidian에 플러그인이 있습니다. 위키 콘텐츠로부터 직접 프레젠테이션을 생성하는 데 유용합니다.
- **Dataview:** 페이지 프론트매터에 대해 쿼리를 실행하는 Obsidian 플러그인. LLM이 위키 페이지에 YAML 프론트매터(태그, 날짜, 소스 개수)를 추가한다면, Dataview로 동적 표와 목록을 생성할 수 있습니다.
- 위키는 단지 마크다운 파일들의 git 저장소일 뿐입니다. 버전 이력, 브랜칭, 협업을 공짜로 얻습니다.

왜 이것이 작동하는가(Why this works)

지식베이스를 유지하는 데 있어 지루한 부분은 읽거나 사고가 아니라 부기(bookkeeping)입니다 — 상호참조 업데이트, 요약물 최신 상태로 유지, 새 데이터가 기존 주장과 모순될 때 기록, 수십 개 페이지에 걸친 일관성 유지. 사람들이 위키를 포기하는 이유는 유지관리 부담이 가치보다 더 빨리 커지기 때문입니다. LLM은 지루해하지 않고, 상호참조 업데이트를 잊지 않으며, 한 번에 15개 파일을 다룰 수 있습니다. 유지관리 비용이 거의 0에 가깝기 때문에 위키가 계속 유지관리 상태를 유지합니다.

사람의 역할은 소스를 큐레이션하고, 분석을 지휘하고, 좋은 질문을 던지고, 그 모든 것이 무엇을 의미하는지 생각하는 것입니다. LLM의 역할은 그 나머지 모든 것입니다.

이 아이디어는 정신적으로 Vannevar Bush의 Memex(1945)와 관련이 있다고 언급합니다 — 문서 사이의 연상적 흔적(associative trails)을 가진 개인적이고 큐레이션된 지식 저장소입니다. Bush의 비전은 웹이 실제로 발전한 방향보다는 이 패턴에 더 가까웠습니다: 사적이고, 능동적으로 큐레이션되며, 문서 간의 연결이 문서 자체만큼이나 가치 있는 방식입니다. Bush가 해결하지 못했던 부분은 "누가 유지관리를 하는가"였는데, LLM이 그것을 처리합니다.

마무리 노트(Note)

이 문서는 의도적으로 추상적으로 작성되었습니다. 특정 구현이 아니라 아이디어를 설명합니다. 정확한 디렉터리 구조, 스키마 관례, 페이지 형식, 도구는 모두 사용자의 도메인, 취향, LLM 선택에 따라 달라질 것입니다. 위에서 언급된 모든 것은 선택적이고 모듈식입니다 — 유용한 것을 고르고 유용하지 않은 것은 무시하십시오. 예를 들어 소스가 텍스트뿐이라면 이미지 처리는 전혀 필요 없을 수 있습니다. 위키가 충분히 작다면 인덱스 파일만으로 충분하고 검색 엔진은 필요 없을 수 있습니다. 슬라이드 덱에 관심이 없고 그냥 마크다운 페이지만 원할 수도 있습니다. 완전히 다른 출력 형식 세트를 원할 수도 있습니다. 이것을 사용하는 올바른 방법은 자신의 LLM 에이전트와 공유하고 함께 협업

하여 자신의 필요에 맞는 버전을 구체화하는 것입니다. 이 문서의 유일한 임무는 패턴을 전달하는 것이며, 나머지는 LLM이 알아낼 수 있습니다.

이상이 원문의 모든 섹션 내용을 빠짐없이 풀어서 정리한 것입니다. 참고로 이 Gist에는 700개가 넘는 후속 댓글이 달려 있는데, 커뮤니티가 이 패턴을 확장한 다양한 구현체 (Obsidian 플러그인, MCP 서버, 코드베이스용 변형, 프로덕션 운영 사례 등)를 소개하고 있습니다. 혹시 댓글 중 특정 구현 사례(예: 코드 저장소 적용, 프로덕션 규모 운영 경험 등)에 대해 더 자세히 알고 싶으시면 말씀해 주세요.

LLM Wiki

A pattern for building personal knowledge bases using LLMs.

This is an idea file, it is designed to be copy pasted to your own LLM Agent (e.g. OpenAI Codex, Claude Code, OpenCode / Pi, or etc.). Its goal is to communicate the high level idea, but your agent will build out the specifics in collaboration with you.

The core idea

Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM is rediscovering knowledge from scratch on every question. There's no accumulation. Ask a subtle question that requires synthesizing five documents, and the LLM has to find and piece together the relevant fragments every time. Nothing is built up. NotebookLM, ChatGPT file uploads, and most RAG systems work this way.

The idea here is different. Instead of just retrieving from raw documents at query time, the LLM **incrementally builds and maintains a persistent wiki** — a structured, interlinked collection of markdown files that sits between you and the raw sources. When you add a new source, the LLM doesn't just index it for later retrieval. It reads it, extracts the key information, and integrates it into the existing wiki — updating entity pages, revising topic summaries, noting where new data contradicts old claims, strengthening or challenging the evolving synthesis. The knowledge is compiled once and then *kept current*, not re-derived on every query.

This is the key difference: **the wiki is a persistent, compounding artifact**. The cross-references are already there. The contradictions have already been flagged. The synthesis already reflects everything you've read. The wiki keeps getting richer with every source you add and every question you ask.

You never (or rarely) write the wiki yourself — the LLM writes and maintains all of it. You're in charge of sourcing, exploration, and asking the right questions. The LLM does all the grunt work — the summarizing, cross-referencing, filing, and bookkeeping that makes a knowledge base

actually useful over time. In practice, I have the LLM agent open on one side and Obsidian open on the other. The LLM makes edits based on our conversation, and I browse the results in real time — following links, checking the graph view, reading the updated pages. Obsidian is the IDE; the LLM is the programmer; the wiki is the codebase.

This can apply to a lot of different contexts. A few examples:

- **Personal:** tracking your own goals, health, psychology, self-improvement — filing journal entries, articles, podcast notes, and building up a structured picture of yourself over time.
- **Research:** going deep on a topic over weeks or months — reading papers, articles, reports, and incrementally building a comprehensive wiki with an evolving thesis.
- **Reading a book:** filing each chapter as you go, building out pages for characters, themes, plot threads, and how they connect. By the end you have a rich companion wiki. Think of fan wikis like [Tolkien Gateway](#) — thousands of interlinked pages covering characters, places, events, languages, built by a community of volunteers over years. You could build something like that personally as you read, with the LLM doing all the cross-referencing and maintenance.
- **Business/team:** an internal wiki maintained by LLMs, fed by Slack threads, meeting transcripts, project documents, customer calls. Possibly with humans in the loop reviewing updates. The wiki stays current because the LLM does the maintenance that no one on the team wants to do.
- **Competitive analysis, due diligence, trip planning, course notes, hobby deep-dives** — anything where you're accumulating knowledge over time and want it organized rather than scattered.

Architecture

There are three layers:

Raw sources — your curated collection of source documents. Articles, papers, images, data files. These are immutable — the LLM reads from them but never modifies them. This is your source of truth.

The wiki — a directory of LLM-generated markdown files. Summaries, entity pages, concept pages, comparisons, an overview, a synthesis. The LLM owns this layer entirely. It creates pages, updates them when new sources arrive, maintains cross-references, and keeps everything consistent. You read it; the LLM writes it.

The schema — a document (e.g. CLAUDE.md for Claude Code or AGENTS.md for Codex) that tells the LLM how the wiki is structured, what the conventions are, and what workflows to follow when ingesting sources, answering questions, or maintaining the wiki. This is the key configuration file — it's what makes the LLM a disciplined wiki maintainer rather than a generic chatbot. You and

the LLM co-evolve this over time as you figure out what works for your domain.

Operations

Ingest. You drop a new source into the raw collection and tell the LLM to process it. An example flow: the LLM reads the source, discusses key takeaways with you, writes a summary page in the wiki, updates the index, updates relevant entity and concept pages across the wiki, and appends an entry to the log. A single source might touch 10-15 wiki pages. Personally I prefer to ingest sources one at a time and stay involved — I read the summaries, check the updates, and guide the LLM on what to emphasize. But you could also batch-ingest many sources at once with less supervision. It's up to you to develop the workflow that fits your style and document it in the schema for future sessions.

Query. You ask questions against the wiki. The LLM searches for relevant pages, reads them, and synthesizes an answer with citations. Answers can take different forms depending on the question — a markdown page, a comparison table, a slide deck (Marp), a chart (matplotlib), a canvas. The important insight: **good answers can be filed back into the wiki as new pages.** A comparison you asked for, an analysis, a connection you discovered — these are valuable and shouldn't disappear into chat history. This way your explorations compound in the knowledge base just like ingested sources do.

Lint. Periodically, ask the LLM to health-check the wiki. Look for: contradictions between pages, stale claims that newer sources have superseded, orphan pages with no inbound links, important concepts mentioned but lacking their own page, missing cross-references, data gaps that could be filled with a web search. The LLM is good at suggesting new questions to investigate and new sources to look for. This keeps the wiki healthy as it grows.

Indexing and logging

Two special files help the LLM (and you) navigate the wiki as it grows. They serve different purposes:

index.md is content-oriented. It's a catalog of everything in the wiki — each page listed with a link, a one-line summary, and optionally metadata like date or source count. Organized by category (entities, concepts, sources, etc.). The LLM updates it on every ingest. When answering a query, the LLM reads the index first to find relevant pages, then drills into them. This works surprisingly well at moderate scale (~100 sources, ~hundreds of pages) and avoids the need for embedding-based RAG infrastructure.

log.md is chronological. It's an append-only record of what happened and when — ingests, queries, lint passes. A useful tip: if each entry starts with a consistent prefix (e.g. `## [2026-04-02] ingest | Article Title`), the log becomes parseable with simple unix tools — `grep "^## \W[" log.md | tail -5` gives you the last 5 entries. The log gives you a timeline of the wiki's evolution and helps

the LLM understand what's been done recently.

Optional: CLI tools

At some point you may want to build small tools that help the LLM operate on the wiki more efficiently. A search engine over the wiki pages is the most obvious one — at small scale the index file is enough, but as the wiki grows you want proper search. [gmd](#) is a good option: it's a local search engine for markdown files with hybrid BM25/vector search and LLM re-ranking, all on-device. It has both a CLI (so the LLM can shell out to it) and an MCP server (so the LLM can use it as a native tool). You could also build something simpler yourself — the LLM can help you vibe-code a naive search script as the need arises.

Tips and tricks

- **Obsidian Web Clipper** is a browser extension that converts web articles to markdown. Very useful for quickly getting sources into your raw collection.
- **Download images locally.** In Obsidian Settings → Files and links, set "Attachment folder path" to a fixed directory (e.g. raw/assets/). Then in Settings → Hotkeys, search for "Download" to find "Download attachments for current file" and bind it to a hotkey (e.g. Ctrl+Shift+D). After clipping an article, hit the hotkey and all images get downloaded to local disk. This is optional but useful — it lets the LLM view and reference images directly instead of relying on URLs that may break. Note that LLMs can't natively read markdown with inline images in one pass — the workaround is to have the LLM read the text first, then view some or all of the referenced images separately to gain additional context. It's a bit clunky but works well enough.
- **Obsidian's graph view** is the best way to see the shape of your wiki — what's connected to what, which pages are hubs, which are orphans.
- **Marp** is a markdown-based slide deck format. Obsidian has a plugin for it. Useful for generating presentations directly from wiki content.
- **Dataview** is an Obsidian plugin that runs queries over page frontmatter. If your LLM adds YAML frontmatter to wiki pages (tags, dates, source counts), Dataview can generate dynamic tables and lists.
- The wiki is just a git repo of markdown files. You get version history, branching, and collaboration for free.

Why this works

The tedious part of maintaining a knowledge base is not the reading or the thinking — it's the bookkeeping. Updating cross-references, keeping summaries current, noting when new data contradicts old claims, maintaining consistency across dozens of pages. Humans abandon wikis because the maintenance burden grows faster than the value. LLMs don't get bored, don't forget

to update a cross-reference, and can touch 15 files in one pass. The wiki stays maintained because the cost of maintenance is near zero.

The human's job is to curate sources, direct the analysis, ask good questions, and think about what it all means. The LLM's job is everything else.

The idea is related in spirit to Vannevar Bush's Memex (1945) — a personal, curated knowledge store with associative trails between documents. Bush's vision was closer to this than to what the web became: private, actively curated, with the connections between documents as valuable as the documents themselves. The part he couldn't solve was who does the maintenance. The LLM handles that.

Note

This document is intentionally abstract. It describes the idea, not a specific implementation. The exact directory structure, the schema conventions, the page formats, the tooling — all of that will depend on your domain, your preferences, and your LLM of choice. Everything mentioned above is optional and modular — pick what's useful, ignore what isn't. For example: your sources might be text-only, so you don't need image handling at all. Your wiki might be small enough that the index file is all you need, no search engine required. You might not care about slide decks and just want markdown pages. You might want a completely different set of output formats. The right way to use this is to share it with your LLM agent and work together to instantiate a version that fits your needs. The document's only job is to communicate the pattern. Your LLM can figure out the rest.

LLM 위키

LLM을 활용한 개인 지식 기반 구축 패턴.

이 파일은 아이디어 파일로, 자신의 LLM 에이전트(예: OpenAI Codex, Claude Code, OpenCode / Pi 등)에 복사해서 붙여넣기 하도록 설계되었습니다. 목표는 고수준 아이디어를 전달하는 것이지만, 구체적인 내용은 에이전트와 협력하여 구축합니다.

핵심 아이디어

대부분의 사람들이 LLM과 문서를 다루는 경험은 RAG와 비슷합니다: 파일 모음을 업로드하면 LLM이 쿼리 시점에 관련 청크를 불러와서 답변을 생성합니다. 이 방법은 효과가 있지만, LLM은 모든 문제에 대해 처음부터 지식을 다시 발견하는 것입니다. 누적은 없어요. 다섯 개의 문서를 종합해야 하는 미묘한 질문을 하면, LLM은 매번 관련 조각들을 찾아내고 조합해야 합니다. 아무것도 쌓이지 않아요. NotebookLM, ChatGPT 파일 업로드, 그리고 대부분의 RAG 시스템이 이렇게 작동합니다.

여기서는 다른 생각이 있습니다. 쿼리 시점에 원시 문서에서 단순히 가져오는 대신, LLM은 **점진적으로 지속 위키를 구축하고 유지합니다** — 이는 당신과 원시 소스 사이에 위치한 구조화되고 상

호 연결된 마크다운 파일 모음입니다. 새로운 소스를 추가할 때, LLM은 단순히 나중에 검색할 수 있도록 인덱싱하는 것이 아닙니다. 이 자료를 읽고, 핵심 정보를 추출한 뒤 기존 위키에 통합합니다 — 엔티티 페이지를 업데이트하고, 주제 요약물 수정하며, 새로운 데이터가 기존 주장과 모순되는 부분을 지적하고, 진화하는 종합을 강화하거나 도전합니다. 지식은 한 번 컴파일되어 최신 *상태*로 유지되며, 매 쿼리마다 재유도되지 않습니다.

핵심 차이점은 위키가 **지속적이고 복합적인 산물**이라는 점입니다. 교차 참조는 이미 존재합니다. 모순은 이미 지적된 상태입니다. 종합은 이미 당신이 읽은 모든 것을 반영하고 있습니다. 위키는 자료를 추가할수록 질문도 점점 더 풍부해집니다.

위키를 직접 작성하는 일은 거의 없고, LLM이 모든 것을 작성하고 유지합니다. 당신은 소원 찾기, 탐색, 그리고 올바른 질문을 던지는 책임을 맡고 있습니다. LLM은 요약, 교차 참조, 서류 정리, 부기 등 모든 기본적인 작업을 담당하여 시간이 지남에 따라 지식 기반을 실제로 유용하게 만듭니다. 실제로는 한쪽에는 LLM 에이전트를, 다른 쪽에는 Obsidian을 열어둡니다. LLM은 우리의 대화를 바탕으로 편집을 하고, 저는 실시간으로 결과를 살펴보고 링크를 따라가고, 그래프 뷰를 확인하며, 업데이트된 페이지를 읽습니다. Obsidian이 IDE입니다; LLM은 프로그래머이며; 위키는 코드베이스입니다.

이것은 다양한 상황에 적용될 수 있습니다. 몇 가지 예를 들자면:

- **개인적인 것:** 자신의 목표, 건강, 심리학, 자기계발 등을 추적하기 — 일기 기록, 기사, 팟캐스트 노트를 정리하고 시간이 지남에 따라 자신에 대한 구조화된 그림을 구축하는 것.
- **연구:** 몇 주 또는 몇 달에 걸쳐 주제를 깊이 파고드는 것 — 논문, 기사, 보고서를 읽고, 점진적으로 진화하는 논제와 함께 포괄적인 위키를 구축하는 것입니다.
- **책을 읽는 것:** 각 장을 정리하면서 등장인물, 주제, 줄거리 실마리, 그리고 그것들이 어떻게 연결되는지 페이지를 구성하는 것. 끝나면 풍부한 동료 위키가 완성됩니다. [톨킨 게이트웨이](#) 같은 팬 위키를 생각해 보세요 — 수천 개의 서로 연결된 페이지들이 등장인물, 장소, 사건, 언어를 다루며, 수년간 자원봉사자 커뮤니티가 만들어 놓았습니다. 직접 읽으면서 LLM이 모든 교차 참조와 유지보수를 담당하는 방식으로 직접 만들 수도 있습니다.
- **비즈니스/팀:** LLM이 유지보수하는 내부 위키, 슬랙 스레드, 회의 기록, 프로젝트 문서, 고객 전화로 제공됩니다. 아마도 인간이 업데이트를 검토하는 루프에 들어갔을 수도 있습니다. 위키가 최신 상태를 유지하는 이유는 LLM이 팀 내 누구도 원하지 않는 유지보수를 하기 때문입니다.
- **경쟁 분석, 실사, 여행 계획, 강의 노트, 취미 심층 탐구** — 시간이 지나면서 지식을 쌓아 흩어지지 않고 체계적으로 쌓아야 하는 모든 것들입니다.

건축

세 가지 층으로 나뉩니다:

원시 자료 — 당신이 엄선한 자료 모음. 기사, 문서, 이미지, 데이터 파일. 이 문서들은 불변이며, LLM은 이를 읽지만 수정하지는 않습니다. 이것이 당신의 진실의 근원입니다.

위키 — LLM이 생성한 마크다운 파일 디렉터리입니다. 요약, 엔터티 페이지, 개념 페이지, 비교, 개요, 종합. LLM이 이 계층을 완전히 소유하고 있습니다. 페이지를 생성하고, 새로운 소스가 도착하면 업데이트하며, 교차 참조를 유지하고, 모든 것이 일관성을 유지합니다. 당신은 그것을 읽는다; LLM이 작성합니다.

스키마 — 문서(예: Claude Code의 CLAUDE.md 또는 Codex의 AGENTS.md)는 LLM에 위키의 구조, 관례, 소스 수집, 질문 답변, 위키 유지 시 따라야 할 워크플로우를 알려줍니다. 이것이 핵심 구성 파일이며, LLM을 단순한 챗봇이 아닌 규율 있는 위키 유지관리자로 만드는 요소입니다. 당신과 LLM은 시간이 지나면서 자신의 도메인에 맞는 방식을 찾아가면서 이 부분을 함께 진화시킵니다.

운영

섭취하세요. 새로운 소스를 원시 컬렉션에 넣고 LLM에 처리하라고 명령합니다. 예시 흐름을 들자면: LLM이 소스를 읽고, 핵심 내용을 논의하며, 위키에 요약 페이지를 작성하고, 색인을 업데이트하며, 위키 전반의 관련 엔터티와 개념 페이지를 업데이트하고, 로그에 항목을 추가합니다. 한 출처가 10-15개의 위키 페이지를 넘길 수도 있습니다. 개인적으로 저는 한 번에 한 자료씩 수집하고 참여하는 것을 선호합니다 — 요약을 읽고, 업데이트를 확인하며, LLM이 강조할 부분을 안내합니다. 하지만 감독이 적은 상태에서 여러 소스를 한 번에 배치 인정할 수도 있습니다. 자신의 스타일에 맞는 워크플로우를 개발하고 향후 세션을 위해 스키마에 문서화하는 것은 당신에게 달려 있습니다.

쿼리. 위키에 대해 질문을 하시는 거죠. LLM은 관련 페이지를 검색해 읽고, 인용과 함께 답변을 종합합니다. 답변은 질문에 따라 다양한 형태로 나타날 수 있습니다 — 마크다운 페이지, 비교표, 슬라이드 덱(Marp), 차트(matplotlib), 캔버스 등이 있습니다. 중요한 통찰은 **좋은 답변은 위키에 새 페이지로 다시 업로드할 수 있다는 점입니다.** 요청한 비교, 분석, 발견한 연결고리 — 이런 것들은 가치 있으며 채팅 기록 속에 사라져서는 안 됩니다. 이렇게 하면 탐구가 수집된 자료처럼 지식 기반에 쌓일 수 있습니다.

보트. 주기적으로 LLM에게 위키 건강 확인을 요청하세요. 다음 항목을 찾아보세요: 페이지 간 모순, 최신 출처가 뒤쳐졌다는 오래된 주장, 인바운드 링크가 없는 고아 페이지, 중요한 개념이 언급되었지만 별도의 페이지가 없는 경우, 누락된 교차 참조, 웹 검색으로 메울 수 있는 데이터 공백. LLM은 새로운 질문과 찾아볼 새로운 자료를 제안하는 데 능숙합니다. 이렇게 하면 위키가 성장하면서 건강하게 유지됩니다.

색인 작성 및 기록

두 개의 특수 파일이 LLM(그리고 당신)이 위키를 탐색하는 데 도움을 줍니다. 이들은 서로 다른 목적을 가지고 있습니다:

index.md 내용 중심입니다. 위키에 있는 모든 것을 나열한 카탈로그로, 각 페이지에는 링크, 한 줄 요약, 그리고 선택적으로 날짜나 출처 수 같은 메타데이터가 포함되어 있습니다. 카테고리별(엔터티, 개념, 출처 등)로 정리되어 있습니다. LLM은 매번 입력할 때마다 업데이트합니다. 쿼리에 답변할 때, LLM은 먼저 인덱스를 읽어 관련 페이지를 찾고, 그 다음에 그 페이지를 깊이 파고듭니다.

이 방법은 중간 규모(~100개의 소스, ~수백 페이지)에서 놀라울 정도로 잘 작동하며, 임베딩 기반 RAG 인프라의 필요성을 피할 수 있습니다.

log.md 연대기적입니다. 이 기록은 언제 무슨 일이 있었는지, 언제 일어났는지 부록 전용으로 기록되어 있습니다 — 섭취, 쿼리, 보풀 통과 기록. 유용한 팁: 각 항목이 일관된 접두사로 시작한다면(예:), 로그는 간단한 유닉스 도구로 해석 가능해집니다 — 마지막 5개의 항목을 제공합니다. 로그는 위키의 진화 타임라인을 제공하며, LLM이 최근에 이루어진 일을 이해하는 데 도움을 줍니다.## [2026-04-02] ingest | Article Titlegrep "^## W[" log.md | tail -5

선택사항: CLI 도구

언젠가는 LLM이 위키에서 더 효율적으로 작동할 수 있도록 돕는 작은 도구를 만들고 싶을 수도 있습니다. 위키 페이지 전체를 검색하는 것이 가장 명확한 방법입니다 — 소규모에서는 인덱스 파일만으로도 충분하지만, 위키가 커질수록 적절한 검색이 필요합니다. [QMD](#)는 좋은 선택입니다: 마크다운 파일을 위한 로컬 검색 엔진으로, BM25/벡터 검색과 LLM 리랭킹 하이브리드 기능을 모두 기기 내에서 지원합니다. CLI(LLM이 직접 사용할 수 있도록)와 MCP 서버(LLM이 네이티브 도구로 사용할 수 있도록)를 모두 갖추고 있습니다. 직접 좀 더 간단한 걸 만들 수도 있어요 — LLM은 필요에 따라 순진한 검색 스크립트를 비브코딩하는 데 도움을 줄 수 있습니다.

팁과 요령

- **옵시디언 웹 클리퍼**는 웹 기사를 마크다운으로 변환하는 브라우저 확장 프로그램입니다. 원본 컬렉션에 소스를 빠르게 넣는 데 매우 유용합니다.
- **이미지를 로컬에서 다운로드하세요.** Obsidian 설정에서 파일 및 링크 →에서 "첨부 폴더 경로"를 고정된 디렉터리(예:). 그 다음 설정 → 단축키에서 "다운로드"를 검색하면 "현재 파일에 대한 첨부파일 다운로드"를 찾아 단축키(예: Ctrl+Shift+D)에 할당하세요. 기사를 클리핑한 후 단축키를 누르면 모든 이미지가 로컬 디스크로 다운로드됩니다. 이것은 선택 사항이지만 유용하며, LLM이 URL에 의존하지 않고 이미지를 직접 보고 참조할 수 있게 해줍니다. LLM은 인라인 이미지로 마크다운을 한 번에 읽을 수 없다는 점에 유의하세요 — 우회 방법은 LLM이 먼저 텍스트를 읽고, 참조된 이미지 일부나 전부를 따로 확인해 추가 맥락을 얻는 것입니다. 조금 불편하지만 꽤 잘 작동합니다.raw/assets/
- **Obsidian의 그래프 뷰**는 위키의 형태를 가장 잘 파악하는 방법입니다 — 무엇이 무엇과 연결되어 있는지, 어떤 페이지가 허브인지, 어떤 페이지가 고아인지 알 수 있습니다.
- **Marp**는 마크다운 기반 슬라이드 덱 형식입니다. Obsidian에는 그런 플러그인이 있습니다. 위키 콘텐츠에서 직접 프레젠테이션을 생성할 때 유용합니다.
- **Dataview**는 페이지 프론트매터를 통해 쿼리를 실행하는 Obsidian 플러그인입니다. LLM이 위키 페이지에 YAML 프론트매터(태그, 날짜, 소스 수를 포함)를 추가하면, Dataview는 동적 테이블과 리스트를 생성할 수 있습니다.
- 위키는 단지 마크다운 파일의 git 저장소일 뿐입니다. 버전 히스토리, 분기, 협업을 무료로 제공합니다.

왜 이런 방법이 효과가 있는지

지식 기반을 유지하는 데 있어 지루한 부분은 읽거나 생각하는 것이 아니라 장부 기록입니다. 교차 참조 업데이트, 요약 유지, 새로운 데이터가 기존 주장과 모순될 때 기록, 수십 페이지에 걸친 일관성 유지. 인간이 위키를 버리는 이유는 유지 관리 부담이 가치보다 더 빨리 늘어나기 때문입니다. LLM은 지루하지 않고, 교차 참조를 업데이트하는 것을 잊지 않으며, 한 번에 15개의 파일을 다룰 수 있습니다. 위키가 유지보수 유지보수 비용은 거의 0에 가깝기 때문입니다.

인간의 역할은 자료를 선별하고, 분석을 이끌며, 좋은 질문을 던지고, 그 모든 의미를 생각하는 것입니다. LLM의 역할은 그 외의 모든 것입니다.

이 아이디어는 Vannevar Bush의 Memex(1945)와 정신적으로 연결되어 있는데, 이는 문서 간에 연관된 흔적이 있는 개인적이고 큐레이션된 지식 저장소입니다. 부시의 비전은 웹이 되었던 것보다는 이와 더 가까웠다: 사적이고 적극적으로 큐레이션되었으며, 문서 간의 연결도 문서 자체만큼이나 가치 있는 것이었다. 그가 해결하지 못한 부분은 누가 유지보수를 하느냐였다. LLM이 그 부분을 처리합니다.

주석

이 문서는 의도적으로 추상적입니다. 구체적인 구현이 아니라 아이디어를 설명하는 것입니다. 정확한 디렉터리 구조, 스키마 관습, 페이지 형식, 도구 — 이 모든 것은 귀하의 도메인, 선호도, 그리고 선택한 LLM에 따라 달라집니다. 위에서 언급한 모든 것은 선택 사항이고 모듈화됩니다 — 유용한 것은 고르고, 그렇지 않은 것은 무시하세요. 예를 들어, 소스가 텍스트 전용일 수 있으니 이미지 처리가 전혀 필요하지 않습니다. 위키가 작아서 인덱스 파일만 있으면 충분할 수도 있고, 검색 엔진이 필요 없을 수도 있습니다. 슬라이드 텍에는 관심이 없고 마크다운 페이지만 원할 수도 있습니다. 완전히 다른 출력 포맷을 원할 수도 있습니다. 이를 올바르게 사용하는 방법은 LLM 에이전트와 공유하고 함께 작업하여 귀하의 필요에 맞는 버전을 인스턴스화하는 것입니다. 문서의 유일한 역할은 패턴을 전달하는 것입니다. 나머지는 LLM이 알아서 해줄 거예요.

LLM Wiki를 위해서 알아야 하는 Obsidian의 기능들을 모두 자세히 정리해서 알려 줘

LLM Wiki 구축을 위해 알아야 할 Obsidian 기능 총정리

LLM Wiki 패턴에서 Obsidian은 "위키를 들여다보는 뷰어(reading environment)" 역할을 합니다. 카파시의 표현을 빌리면 Obsidian은 IDE이고, LLM은 프로그래머이며, 위키는 코드베이스입니다. 즉 사람은 Obsidian으로 결과를 읽고 탐색만 하고, 실제 파일 편집은 LLM 에이전트가 담당하는 구조입니다. 이 관점에서 알아야 할 기능들을 카테고리별로 정리해 드리겠습니다.

1. 코어 플러그인 (기본 내장, 켜기만 하면 됨)

Obsidian 설치 후 **Settings** → **Core plugins**에서 아래 항목들이 켜져 있는지 확인하셔야 합니다. Obsidian 공식 도움말은 Backlinks, Graph view, Outgoing links, Search, Properties 같은 코어 플러그인들을 안내하고 있습니다.

- **Graph view (그래프 뷰):** 위키 전체를 노드-엣지 그래프로 시각화합니다. 위키가 커질수록 가장 중요한 기능인데, 어떤 페이지가 인바운드 링크를 많이 받는 "허브" 개념인지, 어떤 페이지가 아무 데도 연결 안 된 "고아 페이지"인지 한눈에 파악할 수 있습니다. 그래프 뷰는 인바운드 링크가 많은 페이지(위키에서 가장 중요한 개념), 고아 페이지(다음 링크 대상 후보), 지식이 주제별로 어떻게 군집을 이루는지 보여줍니다.
 - **Local Graph(로컬 그래프):** 특정 페이지 하나를 중심으로 그 주변 연결만 보여주는 서브 기능입니다. Depth 1로 설정하면 현재 노트에 직접 연결된 노트만 보여주고, Depth를 올리면 그 이웃 노트에 연결된 노트까지 확장해서 보여줍니다. 이 depth는 Local Graph 패널의 Filters에서 조절할 수 있습니다. 개념 하나씩 짚어가며 점검할 때 유용합니다.
- **Backlinks (백링크):** 어떤 페이지를 열었을 때 "이 페이지를 누가 참조하고 있는가"를 하단에 자동으로 보여줍니다. 모든 개념 페이지에서 어떤 소스 페이지가 그 개념을 언급하고 있는지 백링크로 확인할 수 있습니다. LLM이 새 소스를 수집(ingest)할 때마다 관련 페이지들이 서로 연결되는지 검증하는 핵심 도구입니다.
- **Outgoing links (아웃고잉 링크):** 반대로 현재 페이지가 어떤 페이지들을 참조하고 있는지 보여줍니다. 백링크와 함께 보면 위키 내 연결 관계가 양방향으로 파악됩니다.
- **Search (검색):** 전체 볼트(vault) 텍스트 검색. 위키가 작을 때는 이것만으로도 인덱스 파일 없이 원하는 내용을 찾을 수 있습니다.
- **Properties view (프로퍼티스 뷰):** 각 노트 상단의 YAML 프론트매터(태그, 날짜, 소스 개수 등)를 편집 가능한 UI로 보여줍니다. LLM이 페이지마다 메타데이터를 채워 넣는 경우 이 뷰에서 바로 확인·수정할 수 있습니다.
- **Page preview:** 링크에 마우스를 올리면 해당 페이지 내용을 미리보기로 띄워줍니다. 그래프나 백링크를 클릭하지 않고도 빠르게 확인할 때 유용합니다.

2. 내부 링크 문법 — [[wikilinks]]

위키 페이지 간 연결은 [[페이지 이름]] 형태의 위키링크로 이루어집니다. Obsidian은 노

트 간 내부 링크를 지원하며, [[Three laws of motion]]처럼 위키링크 형태로 쓰거나 [Three laws of motion](Three laws of motion.md)처럼 마크다운 링크 형태로도 쓸 수 있습니다. LLM에게 스키마 파일(CLAUDE.md)에서 "새 개념을 언급할 때는 반드시 [[개념명]] 형식으로 링크하라"고 지시해두면, 이 링크들이 그래프 뷰와 백링크의 재료가 됩니다.

3. 커뮤니티 플러그인 (별도 설치 필요)

Settings → **Community plugins** → **Browse**에서 검색해 설치합니다.

- **Dataview**: 페이지 프론트매터를 대상으로 쿼리를 실행하는 플러그인입니다. LLM이 위키 페이지에 태그, 날짜, 소스 개수 같은 YAML 프론트매터를 추가한다면, Dataview로 동적 표와 목록을 생성할 수 있습니다. 예를 들어 `status: published, confidence: > 0.7, tags: [physics]` 같은 조건으로 페이지를 필터링해 표로 뽑아낼 수 있습니다.
- **Marp**: 마크다운을 슬라이드 덱으로 렌더링하는 플러그인입니다. 위키 콘텐츠로부터 직접 프레젠테이션을 생성하는 데 유용합니다. 위키에 쌓인 종합(synthesis) 페이지를 그대로 발표 자료로 변환하고 싶을 때 씁니다.
- **Obsidian Web Clipper (브라우저 확장, 앱 내 플러그인 아님)**: 웹 아티클을 한 번의 클릭으로 마크다운으로 변환해 `raw/` 폴더에 저장합니다. 원본 콘텐츠와 종합된 해석을 명확히 분리하는 것이 중요하며, 원본 소스는 절대 수정하지 않는 불변 계층으로 취급해야 합니다.

4. 이미지/첨부파일 관련 설정

- **Settings** → **Files and links** → **Attachment folder path**: 여기를 `raw/assets/` 같은 고정 경로로 지정해두면 클리핑한 이미지가 흩어지지 않고 한곳에 모입니다.
- **단축키 바인딩**: **Settings** → **Hotkeys**에서 "Download attachments for current file"을 검색해 단축키(예: `Ctrl+Shift+D`)를 지정하면, 아티클을 클리핑한 뒤 단축키 한 번으로 이미지가 전부 로컬 디스크로 다운로드됩니다. 깨질 수 있는 외부 URL 대신 LLM이 직접 이미지를 참조할 수 있게 해주는 설정입니다.

5. 볼트(Vault) 설정 방식

- **Open folder as vault**: `wiki/` 폴더를 Obsidian 볼트로 바로 열 수 있습니다 — 파일을 따로 импорт하거나 변환할 필요 없이 Obsidian이 폴더를 그대로 읽습니다. 즉 LLM 에이전트(Claude Code)가 작업하는 디렉터리를 그대로 Obsidian 볼트로 지정하면 됩니다.

6. 워크플로우 상의 역할 구분 (기능이라기보다 사용 철학)

Obsidian은 백링크, 그래프 뷰, 검색을 렌더링하고 파일이 디스크에서 바뀔 때마다 실시간으로 갱신해주는, 위키를 위한 IDE 역할을 합니다. 사람은 항상 읽기 모드에 있고 편집 모드에는 있지 않습니다 — 브라우징하고, 링크를 따라가고, 그래프를 확인하고, 읽기만 하며 위키 파일을 직접 편집하지 않습니다. 이 원칙을 지켜야 LLM이 소유한 위키 계층의 일관성이 깨지지 않습니다.

7. 위키가 성장했을 때 참고할 수 있는 확장 기능들

위키가 수백 페이지 규모로 커지면 다음과 같은 커뮤니티 확장이 도움이 됩니다 (필수는 아니고 선택사항):

- **Karpathy LLM Wiki 전용 커뮤니티 플러그인**: 엔티티/개념 페이지와 대화형 질의를 지원하는 다중 페이지 지식 생성 기능을 제공하며, Anthropic·OpenAI·Ollama·LM Studio 등 다양한 LLM 프로바이더를 연결할 수 있습니다. 카파시의 원본 아이디어를 자동화 파이프라인으로 구현한 플러그인입니다.
- **로컬 검색 도구(qmd 등)**: 위키가 100페이지를 넘어서면 인덱스 파일만으로 부족해질 수 있는데, 이때 하이브리드 BM25/벡터 검색을 온디바이스로 제공하는 CLI/MCP 도구를 붙이면 검색 정확도가 올라갑니다.

정리 — 최소 필요 설정 체크리스트

구분	항목	설정 위치
필수(코어)	Backlinks, Graph view, Outgoing links, Search, Properties	Settings → Core plugins
필수(설정)	Attachment folder path → raw/assets/	Settings → Files and links
권장(단축키)	Download attachments for current file	Settings → Hotkeys
선택(커뮤니티)	Dataview	Settings → Community plugins
선택(커뮤니티)	Marp	Settings → Community plugins

구분	항목	설정 위치
선택(확장)	Obsidian Web Clipper	브라우저 확장 프로그램 설치
선택(고급)	Karpathy LLM Wiki 플러그인 / 로컬 검색 (qmd)	Settings → Community plugins / CLI 설치

핵심만 요약하면, **코어 플러그인 5개(백링크·그래프뷰·아웃고잉링크·검색·프로퍼티스)만** **켜져 있으면 카파시가 설명한 최소 요건은 충족**됩니다. Dataview와 Marp는 위키가 성숙해지면서 표나 슬라이드 같은 부가 산출물이 필요할 때 추가하시면 됩니다.

Dataview와 Marp를 처음부터 추가해도 상관은 없지?

네, 처음부터 추가하셔도 전혀 문제없습니다.

- **Dataview와 Marp는 코어 플러그인이 아니라 완전히 독립적인 커뮤니티 플러그인**이라서, 다른 기능들과 충돌하거나 위키 구조 자체에 영향을 주지 않습니다. 커놓아도 아무 일도 안 하다가 필요할 때(YAML 프론트매터를 채우거나, 슬라이드를 만들 때) 쓰면 됩니다.
- 오히려 나중에 추가하려고 하면 "이 페이지들에 프론트매터가 없어서 Dataview 쿼리가 안 돌아간다"는 식으로 소급 작업이 필요해질 수 있는데, 처음부터 CLAUDE.md 스키마에 "모든 페이지에 YAML 프론트매터로 tags, date, source_count 등을 포함하라"고 규칙을 넣어두면 나중에 Dataview를 쓸 때 훨씬 매끄럽습니다.
- 리소스 부담도 거의 없습니다 — 둘 다 가벼운 렌더링/쿼리 플러그인이라 볼트 성능에 영향을 주지 않습니다.

정리하면, **미리 켜두고 스키마에 관련 규칙만 반영해두는 것이 오히려 더 유리한 선택**입니다. 순서 걱정 없이 처음부터 함께 설치하시면 됩니다.